

Uniswap v4 Hooks Audit



Uniswap

March 2, 2026

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	6
Privileged Roles	6
High Severity	7
H-01 Split Swaps Give A Cheaper Price	7
Low Severity	7
L-01 Effective Price For A Corrective Swap Prices Can Far Exceed The Boundary Price	7
Notes & Additional Information	8
N-01 Unnecessary Cast	8
N-02 Possible Duplicate Event Emission	8
N-03 Unused State Variable	9
N-04 Custom Errors in require Statements	9
Conclusion	11
Appendix	12
Issue Classification	12

Summary

Type	DeFi	Total Issues	6 (3 resolved, 1 partially resolved)
Timeline	From 2026-02-09 To 2026-02-13	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	1 (1 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	1 (0 resolved, 1 partially resolved)
		Notes & Additional Information	4 (2 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the "uniswap/v4-hooks-internal" repository at the [a78db37](#) commit.

In scope were the following files:

```
src
├── stable
│   ├── StableStableHook.sol
│   ├── base
│   │   └── FeeConfiguration.sol
│   ├── libraries
│   │   └── FeeCalculation.sol
│   └── interfaces
│       ├── IStableStableHook.sol
│       └── IFeeConfiguration.sol
```

System Overview

StableStableHook is a Uniswap v4 hook designed to provide dynamic fee management for stable/stable token pairs (ex USDC/USDT). The hook aims to create a market maker style fee structure that maintains a consistent bid/ask spread around a reference price.

The hook defines an optimal range around a reference price:

- Lower bound: $\text{referencePrice} \times (1 - \text{optimalFee})$
- Upper bound: $\text{referencePrice} / (1 - \text{optimalFee})$

Within the optimal range, fees are dynamically calculated to push effective swap prices towards the boundary prices (upper boundary if buying, lower boundary if selling). A consistent spread is maintained regardless of where the AMM prices sit within the range.

Outside of the optimal range, fees are also dynamically assessed such that the effective swap price is the farthest boundary from the current price. A decaying fee mechanism activates, operating similar to a dutch auction with fees decaying exponentially in time towards a target fee. As a result, trades are discouraged with push price further from reference, while also gradually incentivizing arbitrageurs to correct price back to the optimal range.

The hook maintains per-pool state tracking:

`decayingFeeE12`: Current position on the decay curve `sqrtAmmPriceX96`: AMM price at last swap `blockNumber`: Block number of last swap

This state enables the decay mechanism to continue across swaps and dynamically adjust for price movements.

The hook also enforces controlled pool creation through `initializePool()`, which requires owner authorization and sets fee configuration before the pool exists. Direct pool initialization through `PoolManager` is blocked via the `beforeInitialize` hook.

Security Model and Trust Assumptions

Privileged Roles

- `owner`:
 - Can initialize new pools via `initializePool()`
 - Set initial fee configuration for pools
 - Standard OpenZeppelin `Ownable` privileges
- `configManager`:
 - Can update fee configuration for any pool via `updateFeeConfig()`
 - Can transfer `configManager` role to another address
 - Can renounce role by setting to `address(0)`, permanently locking configuration

The `configManager` can change `referenceSqrtPriceX96`, `optimalFeeE6`, `k`, and `logK` at any time. Configuration updates also reset fee state, potentially creating fee discontinuities. Additionally, a malicious or compromised `configManager` could set adversarial parameters affecting LP returns.

High Severity

H-01 Split Swaps Give A Cheaper Price

The fee for a trade is calculated such that a trade executed at the current AMM price without price impact would pay the same price as the far boundary price when it is outside of the optimal range and towards the boundary of the swap direction when it is within the optimal range. By splitting a single trade into multiple smaller trades, a user pays a smaller fee percentage in each successive trade and pays a lower total fee. This creates an incentive for users to always split their trades to get a better price, up until the gas costs exceed the fee savings from splitting. This results in inconsistent and exploitable pricing for traders and reduced fees for liquidity providers.

Consider either changing the fee architecture to adjust for price impact or disincentivising/preventing multiple similar trades within the same block.

Update: Resolved at commit [fea1a4c](#). The team stated:

caches the square root price at the start of the block to prevent split swaps being cheaper. caches the price instead of the fee to avoid extra storage

Low Severity

L-01 Effective Price For A Corrective Swap Prices Can Far Exceed The Boundary Price

Immediately after the AMM goes out of range, the fee charged for a corrective swap [corresponds to the price offered at the far boundary price](#). However a swapper is gets a worse price than the far boundary price due the price impact. This is an anti-incentive for a trade which rebalances the pool to the optimal range. The liquidity pool will remain unbalanced until the fee decays sufficiently and during this duration swaps towards the reference price would be too expensive for regular users.

Consider accounting for price impact in the fee charged or acknowledging this issue.

Update: Partially Resolved at commit [c49954f](#). The team stated:

| *acknowledged, made it clear in documentation that all calculations are pre price impact*

Notes & Additional Information

N-01 Unnecessary Cast

Within the `StableStableHook` contract, the `uint160(sqrtAmmPriceX96)` cast is unnecessary.

To improve the overall clarity and intent of the codebase, consider removing any unnecessary casts.

Update: Resolved at commit [929b2bf](#). The team stated:

| *set `sqrtAmmPriceX96` to a `uint256` first, then cast to `uint160` at the end*

N-02 Possible Duplicate Event Emission

When a setter function does not check if the value being set is different from the existing one, it becomes possible to set the same value repeatedly, creating a potential for event spamming. Repeated emission of identical events can also confuse off-chain clients.

Within `FeeConfiguration.sol`, the `setConfigManager` sets the `configManager` and emits an event without checking if the value has changed.

Consider adding a check that reverts the transaction if the value being set is identical to the existing one.

Update: Acknowledged, not resolved. The team stated:

| *acknowledged*

N-03 Unused State Variable

Within the `FeeConfiguration` contract, the `Q24` state variable is unused.

To improve the overall clarity and intent of the codebase, consider removing any unused state variables.

Update: Resolved at commit [109e0dd](#). The team stated:

removed unused Q24 variable

N-04 Custom Errors in `require` Statements

Since Solidity [version 0.8.26](#), custom error support has been added to `require` statements. Initially, this feature was only available through the IR pipeline. However, Solidity [version 0.8.27](#) extended support for this feature to the legacy pipeline as well.

Throughout the codebase, multiple instances of `if-revert` statements were identified that could be replaced with `require` statements:

- The `if (!poolKey.fee.isDynamicFee()) { revert MustUseDynamicFee(poolKey.fee); }` statement in `StableStableHook.sol`
- The `if (poolKey.hooks != IHooks(address(this))) { revert InvalidHookAddress(address(poolKey.hooks)); }` statement in `StableStableHook.sol`
- The `if (msg.sender != configManager) revert NotConfigManager(msg.sender)` statement in `FeeConfiguration.sol`
- The `if (_k == 0 || _logK == 0) { revert InvalidKAndLogK(_k, _logK); }` statement in `FeeConfiguration.sol`
- The `if (_logK != expectedLogK) revert InvalidKAndLogK(_k, _logK)` statement in `FeeConfiguration.sol`
- The `if (_optimalFeeE6 > MAX_OPTIMAL_FEE_E6) { revert InvalidOptimalFeeE6(_optimalFeeE6); }` statement in `FeeConfiguration.sol`
- The `if (_referenceSqrtPriceX96 < minBoundedReferenceSqrtPrice || _referenceSqrtPriceX96 >= maxBoundedReferenceSqrtPrice ) { revert`

`InvalidReferenceSqrtPriceX96(_referenceSqrtPriceX96); }` statement in `FeeConfiguration.sol`

For conciseness and gas savings, consider replacing `if-revert` statements with `require` statements.

Update: *Acknowledged, not resolved. The team stated:*

| *acknowledged*

Conclusion

The `StableStableHook` contract implements a dynamic fee mechanism for stable/stable pairs in Uniswap v4, creating bid/ask spreads around a reference price with exponential decay to incentivize price correction. The primary finding concerns the fee calculation model's reliance on preswap AMM price without accounting for price impact. This causes effective prices to deviate from intended boundaries, enables fee avoidance through swap splitting, and creates anti-incentives for corrective arbitrage. The client has acknowledged these tradeoffs and plans to implement block level fee caching to mitigate same block exploitation.

The codebase demonstrates thoughtful design in its controlled pool initialization pattern and Uniswap v4 hook integration. We appreciate the team's responsiveness and engagement with the design recommendations provided.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.